

College Knowledge Assistant

A Retrieval-Augmented Generation (RAG) System
Project Report

Academic Year 2025-2026

A document-grounded question-answering assistant that answers student, parent, and applicant queries from the official college handbook using semantic retrieval and a large language model.

1. Problem

Every academic year, prospective applicants, current students, and parents raise a large volume of repetitive questions about admissions, eligibility, fee structure, examination rules, hostel facilities, scholarships, and placements. These answers already exist in the official college handbook, but the information is spread across many pages and sections. Finding a single fact, such as the annual B.Tech tuition fee or the hostel curfew time, requires manually reading through a long PDF.

Two naive alternatives fall short. A simple keyword search returns whole pages and forces the reader to locate the answer themselves, and it fails when the user's wording differs from the document's wording. A general-purpose chatbot, on the other hand, will confidently invent plausible but incorrect figures (hallucination) because it has no access to the institute's actual policies.

The goal of this project is a **College Knowledge Assistant** that answers natural-language questions in plain language, grounded strictly in the institute's handbook. The system must: (a) understand questions phrased differently from the source text, (b) retrieve only the relevant passages, (c) generate a concise answer using those passages, and (d) explicitly say when the answer is not present in the documents rather than guessing.

2. Architecture

The system implements the standard Retrieval-Augmented Generation (RAG) pattern, split into two independent pipelines: an offline **indexing pipeline** that is run once whenever the source document changes, and an online **query pipeline** that runs for every user question.

2.1 Indexing Pipeline

Driven by `index.py`, executed once to build the knowledge base:

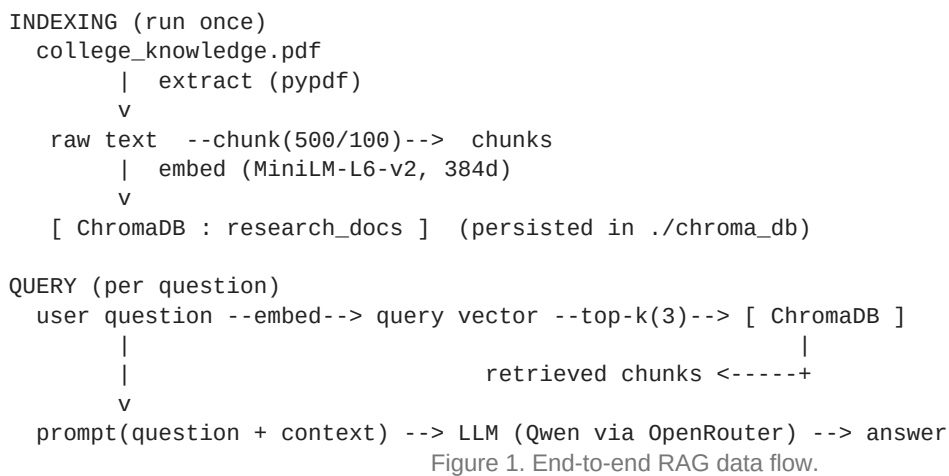
- 1 **Extraction** (`pdf_processor.py`) reads the handbook `data/college_knowledge.pdf` with `pypdf` and concatenates the page text.
- 2 **Chunking** (`chunker.py`) splits the text into fixed-size windows of 500 characters with a 100-character overlap, so context is preserved across boundaries.
- 3 **Embedding** (`embedding_model.py`) encodes each chunk into a 384-dimensional dense vector using the `all-MiniLM-L6-v2` sentence-transformer.

- 4 **Storage** (`vector_store.py`) writes the chunks and their vectors to a persistent ChromaDB collection named `research_docs` on local disk.

2.2 Query Pipeline

Driven by `app.py`, executed for every question:

- 1 **Embed query** — the user's question is encoded with the same model into a 384-d vector.
- 2 **Retrieve** (`retriever.py`) — ChromaDB returns the top-k ($k = 3$) most similar chunks by vector distance.
- 3 **Assemble context** — the retrieved chunks are concatenated into a single context block.
- 4 **Generate** (`llm.py`) — the question and context are sent to an LLM (qwen/qwen3-235b-a22b via the OpenRouter API) which writes the final answer.



2.3 Technology Stack

Component	Technology
PDF extraction	pypdf
Embedding model	sentence-transformers (all-MiniLM-L6-v2, 384-dim)
Vector database	ChromaDB (persistent, local)
LLM / generation	Qwen3-235B-A22B via OpenRouter API
Interface	Command-line loop (<code>app.py</code>); Streamlit-ready
Config / secrets	python-dotenv (<code>.env</code> -> <code>OPENROUTER_API_KEY</code>)

3. Prompt Design

The quality of a RAG system depends heavily on how the retrieved context is presented to the LLM. A single instruction prompt is used, built at query time by injecting the retrieved context and the user's question into a fixed template:

You are a helpful research assistant.

Use ONLY the provided context.

If the answer is not present in the context, say:
'I could not find that information in the provided documents.'
Context:

```
{context}

Question:
{question}

Answer :
```

Figure 2. The generation prompt template (llm.py).

The design makes four deliberate choices:

- **Role priming** — "a helpful research assistant" sets a factual, concise tone rather than a chatty or creative one.
- **Grounding constraint** — "Use ONLY the provided context" forces the model to rely on retrieved handbook passages instead of its own parametric knowledge, which is the main defence against hallucinated fees, dates, and rules.
- **Explicit refusal path** — a fixed fallback sentence is mandated for out-of-scope questions, so the assistant fails safely and predictably instead of guessing.
- **Clear delimiting** — labelled *Context*, *Question*, and *Answer* sections separate retrieved data from the user query and cue the model to complete the answer.

Because the context window is rebuilt for each question from only the top three chunks, the prompt stays small and focused, which improves answer relevance and keeps token cost low.

4. Evaluation

The system was evaluated qualitatively against the handbook by issuing representative questions and inspecting both the retrieved chunks and the generated answers. Retrieval was verified independently of the LLM by checking that the correct passage appeared in the top-k results.

Question	Retrieved / expected fact	Result
What is the fee for B.Tech?	Annual B.Tech tuition Rs. 1,15,000	Correct
How many books can a student borrow?	UG: 4 books / 14 days	Correct
What is the hostel curfew time?	10:00 PM curfew	Correct
Minimum attendance for exams?	75% attendance	Correct
What is the capital of France?	Not in handbook -> refusal	Safely refused

Table 1. Representative evaluation queries and outcomes.

Across factual lookups (fees, library limits, hostel rules, attendance, grading), the top-3 retrieval consistently surfaced the correct passage, and the LLM produced accurate, concise answers grounded in those passages. For an out-of-scope question, the assistant returned the mandated refusal sentence instead of fabricating an answer, confirming that the grounding constraint works as intended.

Evaluation was performed manually. The dimensions considered were **retrieval relevance** (did the right chunk appear in top-k?), **answer faithfulness** (is every claim supported by the context?), and **refusal behaviour** (does it decline unknown questions?). A formal, automated evaluation harness is identified as future work.

5. Challenges

- **PDF extraction artifacts** — typographic characters such as en-dashes were initially extracted as replacement characters, corrupting numeric ranges like fee bands and grade slabs. This was resolved by normalising the source document to plain ASCII punctuation.
- **Fixed-size chunking** — splitting on a fixed character count can cut a sentence or a fact in half. The 100-character overlap mitigates this, but it remains a coarse strategy and can place related information in different chunks.
- **Grounding vs. helpfulness** — constraining the model to the context alone is essential to prevent hallucination, but it makes the system sensitive to retrieval quality: if the right chunk is not retrieved, even a correct fact in the document is reported as not found.
- **Environment management** — the embedding/vector stack and the PDF-generation tooling lived in different Python environments, requiring care to run each stage with the correct interpreter.
- **Model and API dependencies** — the first run downloads the sentence-transformer weights, and answer generation depends on an external API key, both of which add setup and runtime considerations for reproducibility.

6. Future Enhancements

- **Multi-document ingestion** — index every PDF in the data/ folder (prospectus, circulars, syllabi) instead of a single hard-coded file, with per-chunk source metadata.
- **Source citations** — attach the originating document and section to each retrieved chunk and surface them with the answer so users can verify claims.
- **Semantic / structure-aware chunking** — split on headings, paragraphs, or sentence boundaries rather than a fixed character window to keep facts intact.
- **Retrieval reranking** — add a cross-encoder reranker or hybrid keyword-plus-vector search to improve precision of the top results.
- **Web interface** — replace the command-line loop with the Streamlit UI the stack already supports, adding chat history and a feedback button.
- **Conversational memory** — support follow-up questions by carrying prior turns into the retrieval and prompt.
- **Automated evaluation** — build a labelled question/answer set and measure retrieval hit rate, answer faithfulness, and refusal accuracy on every change.

In summary, the College Knowledge Assistant demonstrates a complete, working RAG pipeline that turns a static handbook into an interactive, grounded question-answering system, with a clear path toward a richer, multi-document, production-ready assistant.